



Complete Hadoop Installation + MapReduce Practical (100% Working)

Using Apache Hadoop on Ubuntu/Linux in pseudo-distributed mode.



PART 1 — Install Java 8

Install Java 8

```
sudo apt update  
sudo apt install openjdk-8-jdk -y
```

Select Java 8

For **java**

```
sudo update-alternatives --config java
```

Choose:

```
/usr/lib/jvm/java-8-openjdk-amd64/bin/java
```

For **javac**

```
sudo update-alternatives --config javac
```

Choose:

```
/usr/lib/jvm/java-8-openjdk-amd64/bin/javac
```

Verify

```
java -version
```

```
javac -version
```

Both must show Java 8.

PART 2 — Install SSH

```
sudo apt install openssh-server -y
```

Start SSH:

```
sudo service ssh start
```

Enable SSH:

```
sudo systemctl enable ssh
```

PART 3 — Configure Passwordless SSH

Generate key:

```
ssh-keygen -t rsa -P ""
```

Press Enter for all prompts.

Add key:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Permission:

```
chmod 0600 ~/.ssh/authorized_keys
```

Test:

```
ssh localhost
```

Type:

```
yes
```

PART 4 — Download Hadoop

Go home:

```
cd ~
```

Download:

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

Extract:

```
tar -xvzf hadoop-3.3.6.tar.gz
```

Move:

```
sudo mv hadoop-3.3.6 /usr/local/hadoop
```

PART 5 — Set Environment Variables

Open:

```
nano ~/.bashrc
```

Add at bottom:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
```

Apply:

```
source ~/.bashrc
```

PART 6 — Configure Hadoop

Configure `hadoop-env.sh`

```
nano /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

Set:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

Configure `core-site.xml`

```
nano /usr/local/hadoop/etc/hadoop/core-site.xml
```

Paste:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
```

```
</property>  
</configuration>
```

Configure **hdfs-site.xml**

```
nano /usr/local/hadoop/etc/hadoop/hdfs-site.xml
```

Paste:

```
<configuration>  
  
  <property>  
    <name>dfs.replication</name>  
    <value>1</value>  
  </property>  
  
  <property>  
    <name>dfs.namenode.name.dir</name>  
    <value>file:///home/frozx/hadoopdata/hdfs/namenode</value>  
  </property>  
  
  <property>  
    <name>dfs.datanode.data.dir</name>  
    <value>file:///home/frozx/hadoopdata/hdfs/datanode</value>  
  </property>  
  
</configuration>
```

Configure **mapred-site.xml**

```
nano /usr/local/hadoop/etc/hadoop/mapred-site.xml
```

Paste:

```
<configuration>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>

  <property>
    <name>yarn.app.mapreduce.am.env</name>
    <value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
  </property>

  <property>
    <name>mapreduce.map.env</name>
    <value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
  </property>

  <property>
    <name>mapreduce.reduce.env</name>
    <value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
  </property>

</configuration>
```

Configure **yarn-site.xml**

```
nano /usr/local/hadoop/etc/hadoop/yarn-site.xml
```

Paste:

```
<configuration>

  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>


```

```
<property>
  <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>

</configuration>
```

✓ PART 7 — Create Hadoop Directories

```
mkdir -p ~/hadoopdata/hdfs/namenode
```

```
mkdir -p ~/hadoopdata/hdfs/datanode
```

✓ PART 8 — Format NameNode

```
hdfs namenode -format
```

Must show:

```
Storage directory ... has been successfully formatted
```

✓ PART 9 — Start Hadoop

Start HDFS:

```
start-dfs.sh
```

Start YARN:

```
start-yarn.sh
```

Leave safe mode:

```
hdfs dfsadmin -safemode leave
```



PART 10 — Verify Hadoop

```
jps
```

Expected:

```
NameNode  
DataNode  
SecondaryNameNode  
ResourceManager  
NodeManager
```



PART 11 — MapReduce Practical

Create Working Directory

```
mkdir ~/hadoop_practical  
cd ~/hadoop_practical
```

Create Input File

```
nano log.txt
```

Paste:

```
user1 30  
user2 45
```



```
user1 20
user3 60
user2 10
user3 5
```

Create Java Program

```
nano MaxLoginTime.java
```

Paste:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxLoginTime {

    public static class LogMapper extends Mapper<Object, Text, Text, IntWritable> {

        private Text user = new Text();
        private IntWritable time = new IntWritable();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
```

```

        String[] parts = value.toString().split(" ");

        user.set(parts[0]);
        time.set(Integer.parseInt(parts[1]));

        context.write(user, time);
    }
}

public static class LogReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    private int maxTime = 0;
    private String maxUser = "";

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        if (sum > maxTime) {
            maxTime = sum;
            maxUser = key.toString();
        }
    }

    protected void cleanup(Context context)
        throws IOException, InterruptedException {

        context.write(new Text(maxUser), new IntWritable(maxTime));
    }
}

```

```

    }

    public static void main(String[] args) throws Exception
    {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Max Login Time");

        job.setJarByClass(MaxLoginTime.class);

        job.setMapperClass(LogMapper.class);
        job.setReducerClass(LogReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        job.setNumReduceTasks(1);

        FileInputFormat.addInputPath(job, new Path(args
[0]));
        FileOutputFormat.setOutputPath(job, new Path(args
[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

PART 12 — Compile and Run

Compile:

```
javac -classpath `hadoop classpath` -d . MaxLoginTime.java
```

Create jar:

```
jar -cvf maxlogintime.jar MaxLoginTime*.class
```

Create HDFS input folder:

```
hdfs dfs -mkdir -p /input
```

Upload file:

```
hdfs dfs -put log.txt /input
```

Run job:

```
hadoop jar maxlogintime.jar MaxLoginTime /input /output
```

View output:

```
hdfs dfs -cat /output/part-r-00000
```



Final Output

```
user3    65
```